CPFT User Manual and EPIOT Integration Guide

Charge-Point Forecasting Tool (CPFT) - DriVe2X
Centre for Energy Sustainability, University of Salford

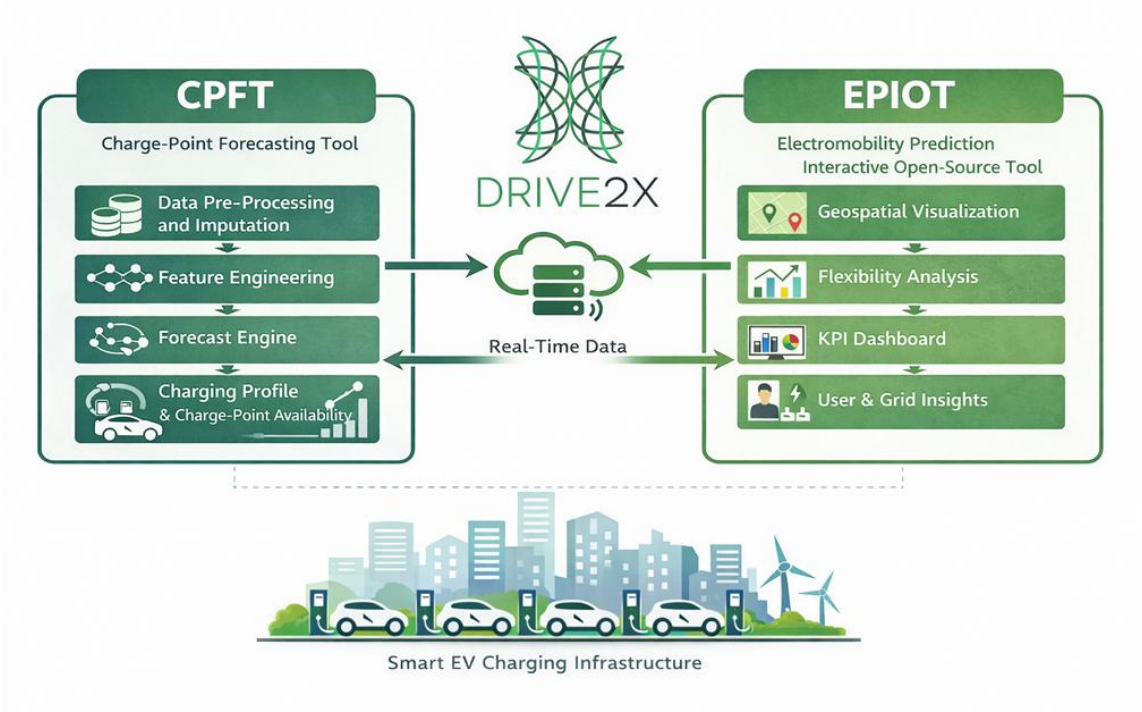
Document purpose: This manual explains how to run CPFT locally and how its outputs are integrated into EPIOT for geospatial visualisation, flexibility analysis, KPI dashboards, and decision support. The focus is documentation of the implemented workflow.

1. System overview

CPFT is a deep learning based forecasting engine that produces two hourly signals for each EV charge-point:

- Availability (binary: in-use / not in-use).
- Charge load (hourly energy delivered, unit: kWh).

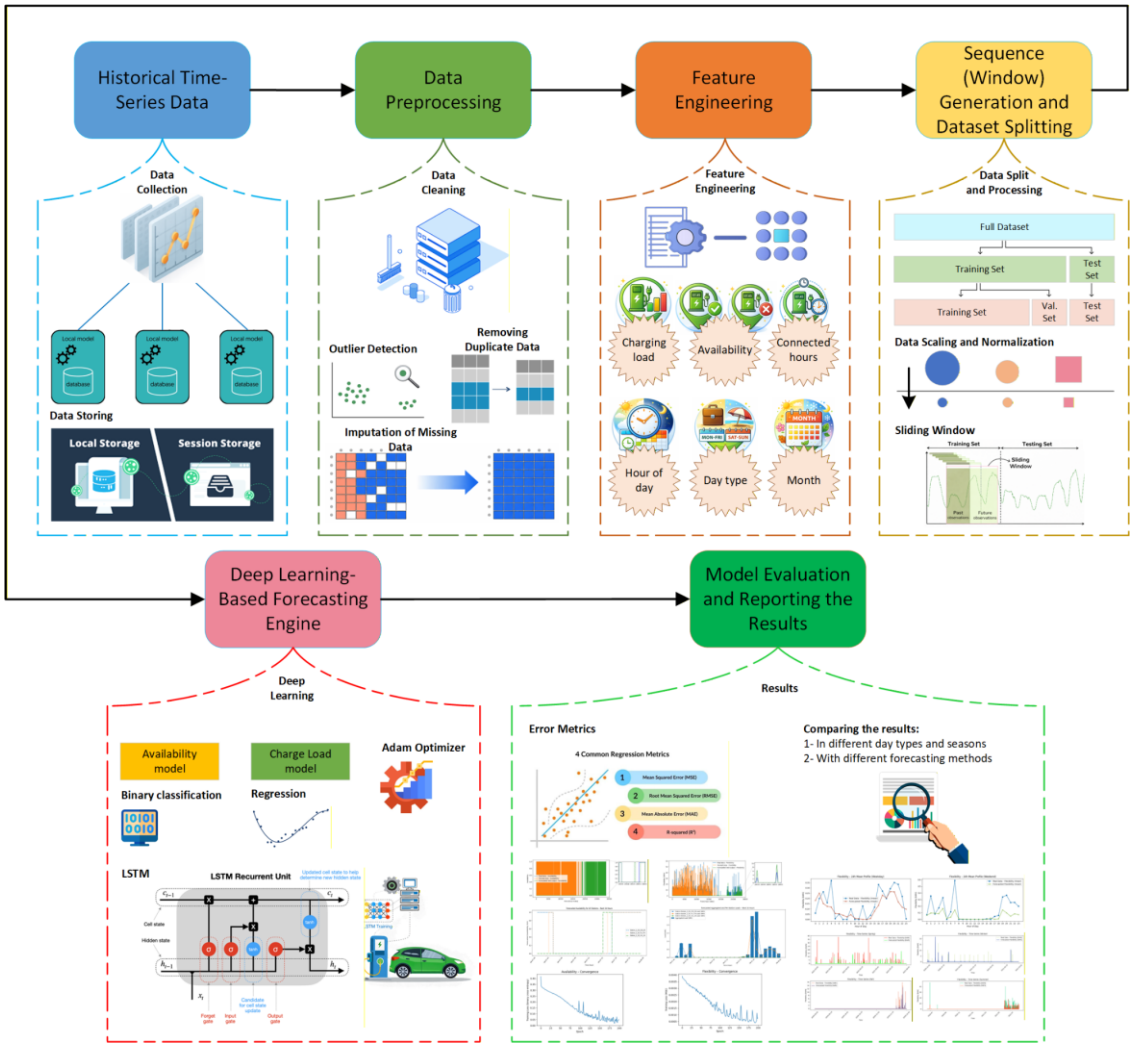
EPIOT consumes CPFT forecasts and combines them with station metadata and operational context to support downstream analytics and visualisation.



Interaction between CPFT and EPIOT within DriVe2X. CPFT produces availability and charge load forecasts that are delivered to EPIOT for visualisation and decision support.

2. CPFT workflow (what the tool does)

CPFT converts session-level charging logs into an hourly time series, engineers temporal features, trains LSTM models, and generates forecasts and reports (plots and Excel exports).



CPFT pipeline: preprocessing, feature engineering, sequence generation, modelling, and reporting.

3. Requirements and installation

Supported environment

- Python 3.9+ (recommended).
- VS Code (recommended) or Jupyter (notebook mode).
- CPU training is supported; GPU is optional for faster training.

Python dependencies (minimum):

- numpy
- pandas
- matplotlib
- scikit-learn
- tensorflow
- openpyxl

Quick start (local):

- Create and activate a virtual environment.
- Install dependencies.
- Run CPFT on a session CSV file.

```
pip install -r requirements.txt
python cpft_pipeline.py \
  --data data/Station_2_39_139_28.csv \
  --outdir outputs \
  --forecast_horizon 168
```

4. Input data specification

CPFT expects a session-level CSV file containing at least the following columns:

Column	Meaning
connectionTime	Charging session start timestamp (UTC or timezone-aware).
disconnectTime	Charging session end timestamp (UTC or timezone-aware).
doneChargingTime	Optional; time charging ended (if different from disconnectTime).
kWhDelivered	Energy delivered during the session (unit: kWh).

If the input file uses an alternative 13-column schema, CPFT can rename columns automatically (as implemented in the notebook/script).

Time zone handling:

- All timestamps are parsed as UTC and the hourly index is built in UTC.

- Excel exports are made Excel-safe by removing timezone metadata.

5. Running CPFT

CPFT can be executed in either notebook mode (Colab/Jupyter) or script mode (VS Code). The VS Code script supports command-line arguments:

Argument	Default	Description
--data	data/Station_2_39_139_28.csv	Path to session CSV (one charge-point or one station file).
--outdir	outputs	Directory to write outputs (models, figures, spreadsheets).
--forecast_horizon	168	Forecast horizon in hours.

Main steps executed by the script:

- Session cleaning and validation.
- Hourly aggregation (availability, occupancy_count, connected_minutes, charge load).
- Outlier handling (train-only z-score clipping) for selected continuous features.
- Feature engineering (calendar + lag + rolling features).
- Sequence (sliding window) generation and chronological split.
- Training: availability model (classification) and charge-load model (regression).
- Evaluation and plots.
- Forecast generation over the requested horizon.
- Excel export of inputs, metrics, predictions, and forecasts.

6. Outputs and file formats

CPFT produces the following outputs (filenames may vary depending on configuration; in VS Code mode they are written under OUTDIR):

Output	Description
Hourly dataset export	kWh_full_data.xlsx (hourly time series created from sessions).
Model checkpoints	best_availability.keras, best_kwh.keras.
Intermediate results	LSTM_all_results.xlsx (metrics, convergence,

	predictions, forecasts).
Seasonal and day-type metrics	season_metrics_*.xlsx, daytype_metrics_*.xlsx.
Comparison table	comparison_table_error_metrics.xlsx (optional baseline comparison).
Consolidated results	EV_forecasting_upgraded_results.xlsx (hourly input, metrics summary, test predictions, and forecast horizon output).

Key forecast sheet for integration:

- In the consolidated Excel, the sheet 'forecast_whole_period' contains hourly forecasts for the selected horizon.

7. Integration into EPIOT

This section describes how CPFT outputs are consumed by EPIOT. The implemented integration is file-based (batch) and can be extended to API-based streaming if required.

7.1 Data contract between CPFT and EPIOT

At minimum, EPIOT requires:

- Station identifier (charge-point ID) to join forecasts with geospatial metadata.
- Hourly timestamp in UTC.
- Availability forecast (probability and/or binary).
- Charge load forecast (unit: kWh).

7.2 File-based integration (current workflow)

Step 1: CPFT runs on a schedule (hourly or daily) and writes forecast outputs to a shared location (local folder, network share, or cloud storage).

Step 2: EPIOT ingests the latest forecast file, parses the 'forecast_whole_period' sheet (or an exported CSV), and merges it with station metadata (including latitude/longitude).

Step 3: EPIOT updates its layers and dashboards:

- Geospatial visualisation (map layers by predicted availability and charge load).
- Flexibility analysis (aggregate by area, time-of-day, or cluster).
- KPI dashboard (utilisation, predicted congestion periods, service coverage).
- User and grid insights (alerts, operational recommendations).

Implementation note: CPFT currently exports Excel. For production ingestion, EPIOT may prefer CSV or Parquet for faster parsing. The same schema can be exported directly from CPFT with minimal changes.

7.2.1 Example ingestion (Python)

The snippet below shows how a downstream tool can read the consolidated Excel and extract the forecast table for ingestion:

```
import pandas as pd

path = 'outputs/EV_forecasting_upgraded_results.xlsx'
df = pd.read_excel(path, sheet_name='forecast_whole_period')
# Expected columns: hour, availability_forecast_prob, availability_forecast_bin,
kWh_forecast_next_whole_period
df = df.rename(columns={
    'hour': 'timestamp_utc',
    'availability_forecast_prob': 'availability_prob',
    'availability_forecast_bin': 'availability_bin',
    'kWh_forecast_next_whole_period': 'charge_load_kwh'
})
# Add cp_id based on file naming or metadata
df['cp_id'] = 'Station_2_39_139_28'
```

7.3 API-based integration (optional extension)

If EPIOT provides a REST endpoint or message broker, CPFT can publish forecasts as JSON. This is not required for the current implementation but is supported by the schema above. Typical patterns include:

- HTTP POST: CPFT posts forecast batches to an EPIOT ingest endpoint.
- Message queue: CPFT publishes forecast records to a topic; EPIOT subscribes.

8. Operational guidance

Scheduling

- Run CPFT inference on a fixed cadence (daily for day-ahead, or hourly for rolling forecasts). Training can be less frequent (weekly or monthly) depending on data drift.

Model management

- Persist best model checkpoints and record model_version in the exported forecast. Keep configuration (seq_len, horizon, feature list, threshold) under version control.

Data quality checks

- Monitor missing timestamps, sudden changes in session volume, and unrealistic charge load spikes. CPFT includes train-only outlier clipping for selected continuous features.

9. Troubleshooting

Issue	Resolution
Missing required columns	Ensure the CSV contains connectionTime, disconnectTime, doneChargingTime (optional), and kWhDelivered. If using the 13-column schema, confirm column order matches the implemented renaming.
Very short datasets	Sequence generation requires at least seq_len + 1 hours after dropping NaNs from lag/rolling features. Increase data history or reduce horizon to lower seq_len.
Timezone issues in Excel	Use the Excel-safe export function that removes timezone metadata. CPFT already applies this in exports.
TensorFlow installation errors	Use a clean virtual environment and install a TensorFlow build compatible with your OS and Python version.
Plots not showing in VS Code	Run in an interactive Python session or ensure matplotlib backend supports your environment. Saved figures can be enabled if needed.

Appendix A. Glossary

Term	Definition
CP	Charge-point (EVSE).
CPFT	Charge-Point Forecasting Tool.
EPIOT	Electromobility Prediction Interactive Open-Source Tool.
Availability	Binary indicator of whether the charge-point is occupied during an hour.
Charge load	Hourly energy delivered (kWh).
Forecast horizon	Number of hours predicted ahead.